SENATI

Escuela de Tecnologías de la Información

SOLUCIONARIO

Solucionario Guía 13

Introducción a la Inteligencia Artificial y al Aprendizaje Automático

Solución paso a paso de los ejercicios propuestos y de las preguntas de reflexión

Escuela	Tecnologías de la Información
Carrera	Ingeniería de Software con Inteligencia Artificial
Módulo formativo	Artificial Intelligence with Machine Learning in Java (Oracle Academy)
Semestre	IV
Semana	13
Documento	Solucionario de los ejercicios propuestos de la Guía 13
Herramientas	Java 21 (JDK) · Visual Studio Code · Extension Pack for Java
Instructor	Rembrandt Ubalde Enríquez
Uso	Material del instructor

Contenido

Indicaciones generales 3
Ejercicio 1. De notas a información 4
Paso 1. Identificar datos, información y decisión 4
Paso 2. Programa 4
Paso 3. Explicación 5
Ejercicio 2. Categorizar los atributos de un producto 6
Paso 1. Clasificar cada atributo 6
Paso 2. Programa 7
Paso 3. Explicación 8
Ejercicio 3. Cambiar la división y la semilla 9
Paso 1. Programa para comparar las configuraciones 9
Paso 2. Análisis 11
Ejercicio 4. KNN con diferentes valores de k 12
Paso 1. Programa 12
Paso 2. Respuesta explicada 13
Ejercicio 5. Identificar el tipo de aprendizaje 14
Respuestas a las preguntas de reflexión 15

(Si el índice aparece vacío, haz clic derecho sobre él y elige «Actualizar campos».)

Indicaciones generales

Cada ejercicio se resuelve en un archivo nuevo dentro de la carpeta `src` del proyecto **Semana13_IA_Java**. Los nombres de los archivos empiezan con `Ej` seguido del número del ejercicio, para no confundirlos con las prácticas.

Ejercicio	Archivo(s)	Necesita
1	`Ej01_NotasEstudiantes.java`	—
2	`Ej02_Productos.java`	—
3	`Ej03_ComparaDivision.java`	Misma lógica de la Práctica 03
4	`Ej04_KNNValoresK.java`	Mismos datos de la Práctica 04
5	Sin código (tabla)	Secciones 2.6 y 2.7 de la guía

Todos los resultados de este solucionario se obtuvieron **ejecutando el código con Java 21**; no son aproximados.

Ejercicio 1. De notas a información

Enunciado: Modifica la Práctica 01 para procesar las notas de 10 estudiantes: calcula el promedio, la nota máxima y mínima, cuántos aprobaron (nota ≥ 10.5) y el porcentaje de aprobados. Si menos del 60 % aprobó, muestra «Programar reforzamiento».

Paso 1. Identificar datos, información y decisión

Nivel	En este ejercicio
Datos	Las 10 notas y los nombres de los estudiantes.
Procesamiento	Un solo bucle `for` que suma, busca el mayor y el menor, y cuenta aprobados.
Información	Promedio, nota máxima y mínima, cantidad y porcentaje de aprobados.
Decisión	Programar reforzamiento si el porcentaje es menor a 60 %.

Paso 2. Programa

Archivo: src/Ej01_NotasEstudiantes.java

```
// Ej01_NotasEstudiantes.java - De datos (notas) a información y decisión
import java.util.Locale;

public class Ej01_NotasEstudiantes {
  public static void main(String[] args) {
    Locale.setDefault(Locale.US);
    // DATOS: notas de 10 estudiantes (escala vigesimal)
    String[] nombres = {"Ana", "Luis", "Rosa", "Pedro", "Carla",
                        "Jorge", "Elena", "Raúl", "Sofía", "Diego"};
    double[] notas = {12.5, 8.0, 15.0, 10.0, 9.5, 17.0, 11.0, 6.5, 13.5, 10.5};

    // PROCESAMIENTO
    double suma = 0;
    int iMax = 0, iMin = 0, aprobados = 0;
    for (int i = 0; i < notas.length; i++) {
      suma += notas[i];
      if (notas[i] > notas[iMax]) iMax = i;
      if (notas[i] < notas[iMin]) iMin = i;
      if (notas[i] >= 10.5) aprobados++;
    }
    double promedio = suma / notas.length;
    double porcentaje = 100.0 * aprobados / notas.length;

    // INFORMACIÓN
    System.out.printf("Promedio del aula : %.2f%n", promedio);
    System.out.printf("Nota más alta     : %.1f (%s)%n", notas[iMax], nombres[iMax]);
    System.out.printf("Nota más baja     : %.1f (%s)%n", notas[iMin], nombres[iMin]);
    System.out.printf("Aprobados         : %d de %d (%.1f %%)%n",
        aprobados, notas.length, porcentaje);

    // DECISIÓN
    if (porcentaje < 60) {
      System.out.println("Decisión: Programar reforzamiento");
    } else {
      System.out.println("Decisión: Rendimiento adecuado, continuar con el sílabo");
    }
  }
}
```

▶ Resultado esperado en la terminal

```
Promedio del aula : 11.35
Nota más alta     : 17.0 (Jorge)
Nota más baja     : 6.5 (Raúl)
Aprobados         : 6 de 10 (60.0 %)
Decisión: Rendimiento adecuado, continuar con el sílabo
```

Paso 3. Explicación

- Se guardan **índices** (`iMax`, `iMin`) en lugar de valores para poder mostrar también el nombre del estudiante.
- El porcentaje se calcula con `100.0 * aprobados / notas.length`. El `100.0` hace que la división sea decimal; con `100 * aprobados / 10` también funcionaría aquí, pero con otros totales Java haría una **división entera**.
- Con estos datos aprueba **exactamente el 60 %**, así que la condición `porcentaje < 60` es falsa y no se programa reforzamiento. Es un buen caso de borde para discutir: si Diego tuviera 10.0 en lugar de 10.5, el porcentaje bajaría a 50 % y sí aparecería «Programar reforzamiento».

Ejercicio 2. Categorizar los atributos de un producto

Enunciado: Crea un `record Producto` con los atributos código, precio, categoría, talla (S/M/L), enOferta y stock. Indica el tipo de dato de cada atributo y muestra: precio promedio, productos por categoría y productos con talla M o superior.

Paso 1. Clasificar cada atributo

Atributo	Tipo de dato	Tipo en Java	Operación válida
código	Identificador	`String`	Ninguna: solo identifica
precio	Numérico continuo	`double`	Promedio, mínimo, máximo
categoría	Categórico nominal	`String`	Contar frecuencias
talla	Categórico ordinal	`enum Talla { S, M, L }`	Comparar (S < M < L)
enOferta	Booleano	`boolean`	Contar verdaderos
stock	Numérico discreto	`int`	Sumar, promedio

Paso 2. Programa

Archivo: src/Ej02_Productos.java

```
// Ej02_Productos.java - Categorización de los atributos de un producto
import java.util.Locale;
import java.util.Map;
import java.util.TreeMap;

public class Ej02_Productos {

  enum Talla { S, M, L }                    // categórico ORDINAL

  record Producto(String codigo,            // identificador (no se usa para aprender)
                  double precio,            // numérico CONTINUO
                  String categoria,         // categórico NOMINAL
                  Talla talla,              // categórico ORDINAL
                  boolean enOferta,         // BOOLEANO
                  int stock) { }            // numérico DISCRETO

  public static void main(String[] args) {
    Locale.setDefault(Locale.US);
    Producto[] productos = {
      new Producto("P01", 49.90, "Polo",     Talla.S, false, 12),
      new Producto("P02", 59.90, "Polo",     Talla.M, true,  20),
      new Producto("P03", 89.90, "Pantalón", Talla.L, false,  7),
      new Producto("P04", 79.90, "Pantalón", Talla.M, true,  15),
      new Producto("P05", 129.90, "Casaca",  Talla.L, false,  4),
      new Producto("P06", 39.90, "Polo",     Talla.L, true,  30),
    };

    System.out.println("Atributo   | Tipo de dato");
    System.out.println("-----------+--------------------------");
    System.out.println("codigo     | Identificador (no se analiza)");
    System.out.println("precio     | Numérico continuo");
    System.out.println("categoria  | Categórico nominal");
    System.out.println("talla      | Categórico ordinal (S < M < L)");
    System.out.println("enOferta   | Booleano");
    System.out.println("stock      | Numérico discreto");

    double suma = 0;
    for (Producto p : productos) suma += p.precio();
    System.out.printf("%nPrecio promedio: S/ %.2f%n", suma / productos.length);

    Map<String, Integer> porCategoria = new TreeMap<>();
    for (Producto p : productos) porCategoria.merge(p.categoria(), 1, Integer::sum);
    System.out.println("Productos por categoría: " + porCategoria);

    System.out.print("Talla M o superior: ");
    for (Producto p : productos) {
      if (p.talla().compareTo(Talla.M) >= 0) System.out.print(p.codigo() + " ");
    }
    System.out.println();
  }
}
```

▶ **Resultado esperado en la terminal**

```
Atributo   | Tipo de dato
-----------+---------------------------
codigo     | Identificador (no se analiza)
precio     | Numérico continuo
categoria  | Categórico nominal
talla      | Categórico ordinal (S < M < L)
enOferta   | Booleano
stock      | Numérico discreto

Precio promedio: S/ 74.90
Productos por categoría: {Casaca=1, Pantalón=2, Polo=3}
Talla M o superior: P02 P03 P04 P05 P06
```

Paso 3. Explicación

- El `enum` guarda el **orden** en que se declaran sus valores. Por eso `p.talla().compareTo(Talla.M) >= 0` es verdadero para M y L.
- La categoría es **nominal**: no se promedia, se cuentan sus frecuencias con un `TreeMap`, que además las ordena alfabéticamente.
- El código del producto no se usa para analizar ni para entrenar un modelo, porque no describe al producto: solo lo identifica.

Ejercicio 3. Cambiar la división y la semilla

Enunciado: En la Práctica 03 cambia la división a 70 % / 30 % y la semilla a 7. ¿Cómo cambian la ecuación, el MAE y el R^2? Explica por qué.

Paso 1. Programa para comparar las configuraciones

Se convierte el flujo de la Práctica 03 en un método `ejecutar(proporcionTrain, semilla)`. Así se comparan las tres configuraciones en una sola ejecución.

Archivo: src/Ej03_ComparaDivision.java

```
// Ej03_ComparaDivision.java - Efecto de la división y la semilla en el modelo
import java.util.ArrayList;
import java.util.Collections;
import java.util.List;
import java.util.Locale;
import java.util.Random;

public class Ej03_ComparaDivision {

  // Entrena y evalúa el mismo flujo de la Práctica 03 con otra división y semilla
  static void ejecutar(double proporcionTrain, long semilla) {
    Random azar = new Random(semilla);
    List<double[]> datos = new ArrayList<>();
    for (int i = 0; i < 30; i++) {
      double horas = Math.round((1 + 9 * azar.nextDouble()) * 10) / 10.0;
      double nota = 5 + 1.4 * horas + azar.nextGaussian() * 1.2;
      nota = Math.round(Math.min(20, Math.max(0, nota)) * 10) / 10.0;
      datos.add(new double[]{horas, nota});
    }
    Collections.shuffle(datos, azar);
    int corte = (int) (datos.size() * proporcionTrain);
    List<double[]> train = datos.subList(0, corte);
    List<double[]> test = datos.subList(corte, datos.size());

    double mx = 0, my = 0;
    for (double[] d : train) { mx += d[0]; my += d[1]; }
    mx /= train.size(); my /= train.size();
    double num = 0, den = 0;
    for (double[] d : train) {
      num += (d[0] - mx) * (d[1] - my);
      den += (d[0] - mx) * (d[0] - mx);
    }
    double b1 = num / den, b0 = my - b1 * mx;

    double mae = 0, ssRes = 0, ssTot = 0, myTest = 0;
    for (double[] d : test) myTest += d[1];
    myTest /= test.size();
    for (double[] d : test) {
      double pred = b0 + b1 * d[0];
      mae += Math.abs(d[1] - pred);
      ssRes += Math.pow(d[1] - pred, 2);
      ssTot += Math.pow(d[1] - myTest, 2);
    }
    System.out.printf("%2.0f/%2.0f  semilla=%-3d train=%2d test=%2d  "
        + "nota = %.3f + %.3f*h  MAE=%.3f  R²=%.3f%n",
        proporcionTrain * 100, (1 - proporcionTrain) * 100, semilla,
        train.size(), test.size(), b0, b1, mae / test.size(), 1 - ssRes / ssTot);
  }

  public static void main(String[] args) {
    Locale.setDefault(Locale.US);
    ejecutar(0.8, 42);     // configuración original de la Práctica 03
    ejecutar(0.7, 42);     // solo cambia la división
    ejecutar(0.7, 7);      // cambia la división y la semilla
  }
}
```

▶ Resultado esperado en la terminal

```
80/20  semilla=42  train=24 test= 6  nota = 4.905 + 1.427*h  MAE=1.587  R²=0.732
70/30  semilla=42  train=21 test= 9  nota = 5.304 + 1.342*h  MAE=1.570  R²=0.830
70/30  semilla=7   train=21 test= 9  nota = 5.845 + 1.234*h  MAE=1.119  R²=0.866
```

Paso 2. Análisis

Configuración	Ecuación	MAE	R^2
80/20, semilla 42 (original)	4.905 + 1.427·h	1.587	0.732
70/30, semilla 42	5.304 + 1.342·h	1.570	0.830
70/30, semilla 7	5.845 + 1.234·h	1.119	0.866

- **Al cambiar solo la división**, el modelo se entrena con 21 estudiantes en lugar de 24. Con otros datos de entrenamiento obtiene otra pendiente y otro intercepto. Además se evalúa con 9 estudiantes distintos, por eso cambian el MAE y el R^2.
- **Al cambiar la semilla**, cambian los propios datos generados y el orden de la mezcla, así que el modelo y la evaluación son otros.
- **Conclusión:** que el R^2 haya subido no significa que 70/30 sea «mejor». Con solo 30 datos, los resultados dependen mucho de **qué** estudiantes caen en la prueba. Por eso, en proyectos reales se repite la evaluación con varias divisiones (**validación cruzada**) y se promedian los resultados.

Ejercicio 4. KNN con diferentes valores de k

Enunciado: En la Práctica 04 clasifica la fruta (172 g, textura 5) con k = 1 y con k = 5. ¿Cambia la respuesta? ¿Por qué conviene que k sea impar?

Paso 1. Programa

Se ordenan las 10 frutas por distancia **una sola vez** y luego se cuentan los votos de los k primeros para cada valor de k.

Archivo: src/Ej04_KNNValoresK.java

```
// Ej04_KNNValoresK.java - ¿Cómo cambia la clasificación según k?
import java.util.Arrays;
import java.util.Comparator;
import java.util.Locale;

public class Ej04_KNNValoresK {

  static double[][] X = {
    {150, 2}, {170, 1}, {140, 3}, {160, 2}, {130, 2},
    {180, 8}, {200, 9}, {190, 7}, {210, 8}, {175, 9}
  };
  static String[] y = {"manzana", "manzana", "manzana", "manzana", "manzana",
                       "naranja", "naranja", "naranja", "naranja", "naranja"};

  static double distancia(double[] a, double[] b) {
    double dPeso = a[0] - b[0];
    double dTextura = (a[1] - b[1]) * 10;
    return Math.sqrt(dPeso * dPeso + dTextura * dTextura);
  }

  public static void main(String[] args) {
    Locale.setDefault(Locale.US);
    double[] fruta = {172, 5};
    Integer[] idx = new Integer[X.length];
    for (int i = 0; i < X.length; i++) idx[i] = i;
    Arrays.sort(idx, Comparator.comparingDouble(i -> distancia(fruta, X[i])));

    System.out.println("Vecinos ordenados por distancia a (172 g, textura 5):");
    for (int j = 0; j < 5; j++) {
      int i = idx[j];
      System.out.printf("  %d. %-8s (%.0f g, textura %.0f)  distancia = %.1f%n",
          j + 1, y[i], X[i][0], X[i][1], distancia(fruta, X[i]));
    }
    System.out.println();
    for (int k : new int[]{1, 3, 5}) {
      int manzanas = 0;
      for (int j = 0; j < k; j++) if (y[idx[j]].equals("manzana")) manzanas++;
      int naranjas = k - manzanas;
      String clase = manzanas > naranjas ? "manzana" : "naranja";
      System.out.printf("k = %d -> votos: %d manzana, %d naranja => %s%n",
          k, manzanas, naranjas, clase.toUpperCase());
    }
  }
}
```

▶ Resultado esperado en la terminal

```
Vecinos ordenados por distancia a (172 g, textura 5):
  1. naranja  (190 g, textura 7)  distancia = 26.9
  2. naranja  (180 g, textura 8)  distancia = 31.0
  3. manzana  (160 g, textura 2)  distancia = 32.3
  4. manzana  (150 g, textura 2)  distancia = 37.2
  5. manzana  (140 g, textura 3)  distancia = 37.7

k = 1 -> votos: 0 manzana, 1 naranja => NARANJA
k = 3 -> votos: 1 manzana, 2 naranja => NARANJA
k = 5 -> votos: 3 manzana, 2 naranja => MANZANA
```

Paso 2. Respuesta explicada

- **Sí cambia.** Con k = 1 y k = 3 la fruta es **naranja**, porque las dos más cercanas son naranjas. Con k = 5 entran tres manzanas que están un poco más lejos y ganan la votación 3 a 2: la respuesta pasa a **manzana**.
- La fruta de 172 g y textura 5 está justo **en la frontera** entre ambos grupos, por eso es sensible al valor de k.
- **k impar:** con dos clases, un k impar evita los **empates**. Con k = 4, por ejemplo, podría haber 2 votos contra 2 y el programa no sabría qué decidir.
- **k muy pequeño** (k = 1) es sensible al ruido: un solo ejemplo mal etiquetado decide. **k muy grande** incluye vecinos lejanos que ya no se parecen. Un buen k se elige probando varios valores con datos de prueba.

Ejercicio 5. Identificar el tipo de aprendizaje

Enunciado: Indica si cada problema es supervisado o no supervisado y si es clasificación, regresión o agrupamiento.

Problema	Tipo de aprendizaje	Tarea	Justificación
a) Predecir el precio de un departamento	Supervisado	Regresión	Hay ventas pasadas con su precio (etiqueta) y se predice un número.
b) Detectar correos spam	Supervisado	Clasificación	Hay correos marcados como spam o no spam; la salida es una categoría.
c) Formar grupos de clientes con gustos parecidos	No supervisado	Agrupamiento	No hay grupos definidos de antemano; el algoritmo los descubre.
d) Predecir la temperatura de mañana	Supervisado	Regresión	Se aprende de días pasados con su temperatura real; la salida es un número.
e) Diagnosticar neumonía en una radiografía	Supervisado	Clasificación	Las radiografías de entrenamiento tienen el diagnóstico del médico; la salida es Sí/No.
f) Organizar noticias por temas sin categorías previas	No supervisado	Agrupamiento	No existen etiquetas; se agrupan las noticias parecidas.

> 💡 **Consejo:** Pregunta clave para decidir: **¿los datos de entrenamiento traen la respuesta correcta?** Si la traen, es supervisado; luego se mira si esa respuesta es un número (regresión) o una categoría (clasificación).

Respuestas a las preguntas de reflexión

1. ¿En qué se diferencia un programa tradicional de uno que aprende?

En un programa tradicional el programador escribe las reglas, por ejemplo `if (horas > 5) nota = 15;`. En la Práctica 03 nadie escribió la relación entre horas y nota: el programa **calculó** la pendiente y el intercepto (`nota = 4.905 + 1.427 * horas`) a partir de 24 ejemplos. Si cambian los datos, el modelo cambia solo, sin modificar el código.

2. ¿Por qué no tiene sentido promediar el distrito?

Porque es un dato **nominal**: sus valores son nombres sin magnitud ni orden. No existe un «distrito promedio» entre Comas y Los Olivos. Con datos nominales solo tiene sentido contar frecuencias o hallar la **moda** (el valor más repetido).

3. ¿Por qué se evalúa con datos de prueba?

Porque lo que importa es cómo funciona el modelo con datos **nuevos**. Si se evalúa con los datos de entrenamiento, el resultado sale demasiado optimista: es como dar un examen con las mismas preguntas que se usaron para practicar. Los datos de prueba simulan el uso real del modelo.

4. Usos de la IA en la comunidad o el trabajo

Respuesta abierta. Ejemplos válidos: predecir la demanda de productos en una bodega (supervisado, regresión); detectar si un pago es fraudulento (supervisado, clasificación); agrupar a los estudiantes según su rendimiento para formar grupos de estudio (no supervisado, agrupamiento). Se espera que el estudiante indique **qué datos** usaría y **qué tipo** de aprendizaje es.